Computer Communications 62 (2015) 23–33

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Improving datacenter throughput and robustness with Lazy TCP over packet spraying

Jie Zhang [a], Dafang Zhang [a,*], Kun Huang [b]

[a] College of Computer Science and Electronics Engineering, Hunan University, Changsha, China
[b] Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China

ARTICLE INFO

Article history:
Received 31 May 2014
Received in revised form 30 December 2014
Accepted 17 February 2015
Available online 25 February 2015

Keywords:
Datacenter network
TCP
Packet spraying
Packet reordering
Link degradation

ABSTRACT

Packet spraying is being recognized as a promising multipath routing approach in datacenter for full utilization of multiple paths. However, existing TCP schemes face serious challenge of robustness when running over packet spraying. They suffer severely from frequent packet reordering and link degradation, resulting in low throughput. To address these issues, this paper proposes Lazy TCP (LTCP), a novel TCP over packet spraying to provide robustness and achieve high throughput. Our basic idea is that TCP should not backoff quickly when receiving congestion signals, as these signals can be easily caused by non-congestion events in packet spraying circumstance. LTCP performs congestion avoidance in a lazy way. First, to mitigate the impact of packet reordering, LTCP delays the time to perform congestion avoidance. As the sender waits more time for the disordered packet, LTCP avoids unnecessary backoff caused by packet reordering. Second, to eliminate the impact of link degradation, LTCP stubbornly maintains data transmission rate. When performing congestion avoidance, LTCP never halves data transmission rate unless identifying successive packet loss. Such design is based on our key observation that degraded link stands a good chance of causing discrete packet loss in a batch of packets. And it effectively eliminates the impact of link degradation. Simulations on NS2 show that LTCP is robust and high-throughput, and improves 2.4 times throughput compared to existing TCP in cases of packet reordering and link degradation.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

1.1. Background and motivation

Datacenter is the key infrastructure of cloud computing and being the footstone of modern Internet service. A Datacenter contains hundreds of thousands of servers hosting a variety of large-scale data processing applications and services such as web search, social collaboration, and scientific computation. Due to the inherently distributed nature of big data processing, the datacenter network (DCN) connecting a mass of servers often becomes the bottleneck. Therefore new DCN topologies, such as FatTree [1] and VL2 [2], have been recently proposed to provide full bisection bandwidth.

In such multi-rooted tree topologies, there are multiple paths between each pair of hosts. Switches usually use an ECMP (Equal Cost Multi-Path) routing scheme, which randomly hashes flows on these paths for load balancing. Over ECMP the transport-layer protocol is typically dominated by TCP. A recent study [3] shows that 99.91% of traffic in the datacenter is TCP traffic. Hence, a high-throughput TCP is desirable as it dominantly determines the performance of DCN.

Recent researches trend to improve TCP throughput by utilizing the dense paths between hosts, and many multipath schemes have been proposed, such as Hedera [4] and MPTCP [5]. These schemes focus on improving bandwidth utilization of the underlying routing paths by various load balancing techniques, and they significantly improve TCP throughput.

Among all multipath schemes, packet spraying is being recognized as a promising one for full utilization of multiple paths and convenient deployment. It uses a per-packet routing mechanism to evenly distribute packets with the same destination among all downstream paths. As a result, it achieves almost perfect load balance and excellent bandwidth utilization of multiple links in a DCN. Recent studies [5,6] show that TCP over packet spraying achieves higher throughput than MPTCP over ECMP. In addition, packet spraying has been widely implemented in commodity switch for many years [7]. So its throughput gain is realistic and can be easily achieved by configuring switch.

* Corresponding author.
E-mail addresses: jiezhang@hnu.edu.cn (J. Zhang), dfzhang@hnu.edu.cn (D. Zhang), huangkun09@ict.ac.cn (K. Huang).

http://dx.doi.org/10.1016/j.comcom.2015.02.011
0140-3664/© 2015 Elsevier B.V. All rights reserved.

However, TCP over packet spraying faces the challenges of throughput and robustness. First, packets sprayed among multiple paths may naturally be in disorder, which causes duplicate ACKs as congestion does. Since TCP takes duplicate ACKs as congestion signals, it cannot distinguish such packet reordering from congestion. As a result, packet reordering can trigger TCP congestion avoidance and decrease its throughput. In addition, low-cost switches are widely used in modern DCNs, leading to frequent link failure. These failed links may break the original symmetric topology of DCN, which aggravates packet reordering [6]. Second, link degradation also occurs frequently in DCNs. Packets sprayed to a degraded link may be possibly delayed or dropped, which in turn enforces TCP to perform continuous congestion avoidance, resulting in low throughput of TCP. A recent study [5] shows that link degradation is the major reason, which impedes the practical applications of packet spraying in DCNs.

Previous TCP approaches cannot solve both two problems of packet reordering and link degradation. Internet-oriented TCP schemes have been proposed, such as TCP-Reno [8], TCP-NewReno [9], and TCP–SACK [10]. But these schemes do not handle packet reordering. TCP–NCR [11] is a robust TCP scheme, but not resilient to link degrading showed by our experiments in Section 5. What is more, our experiment also shows that even DCTCP does not perform well in cases of packet reordering and link degradation. An alternative way is modifying packet spraying on switch to avoid packet reordering and mitigate the impact of link degradation. However, such load-aware packet spraying scheme might be costly to implement. For example, DeTail [12] requires modification on both switches and end hosts' TCP, which leads to high cost of implementation in large-scale DCNs. In this paper, we are motivated to propose a robust TCP scheme, which handles both two problems above and achieves reliable transmission over packet spraying.

1.2. Our approach

Our goal is to design a robust TCP over packet spraying to provide high throughput in abnormal circumstances. Our basic idea is that TCP over packet spraying should not backoff quickly when receiving congestion signals, as these signals can be easily caused by non-congestion events, such as packet reordering and link degradation. We present Lazy TCP (LTCP), which performs congestion avoidance in a lazy way. LTCP is designed as follows:

- To mitigate the impact of packet reordering, the time of triggering congestion avoidance is delayed. TCP performs congestion avoidance as soon as it receives congestion signals, which are usually indicated by three duplicate ACKs. LTCP needs much more duplicate ACKs to trigger congestion avoidance, hence the sender waits more time for the disordered packet. We call such design lazy trigger, and it significantly reduces unnecessary congestion avoidance caused by packet reordering.
- To eliminate the impact of link degradation, the congestion window is stubbornly maintained. When performing congestion avoidance, existing TCP schemes directly halve congestion window to slow down data transmission rate. LTCP first identifies the state of packet loss and then takes proper actions. In case of successive packet loss, LTCP halves congestion window as existing TCP schemes do. In case of discrete packet loss, LTCP maintains congestion window. We call such design stubborn window. It exploits the characteristic of discrete packet loss caused by link degradation, and it effectively eliminates the impact of link degradation.
- To cut down the overhead of packet loss, the max congestion window is set to a fixed value. The flip side of the coin is that LTCP will not backoff properly if discrete packet loss is caused by congestion. As a result, LTCP causes more packet loss on the bottleneck link than existing TCP schemes. To reduce the overhead of packet loss, we set a max congestion window for LTCP. Such design ensures that LTCP forms a limited queue on the bottleneck link. The limited queue not only significantly reduces packet loss on the bottleneck link, but also benefits time sensitive short flows for low latency.

Moreover, LTCP can be implemented easily. LTCP is a host-based scheme, which requires only a little modification on existing TCP schemes. And it directly adopts the widely used RRPS (Round-Robin packet spraying) [7] and DropTail queue implemented in most commodity switches.

1.3. Key contributions

We make three key contributions as follows:

- We observe that just a few degraded links can have a severe impact of TCP throughput in large-scale DCNs. We also make a key observation that link degradation stands a good chance of causing discrete packet loss in a batch of packets. By taking advantage of this observation, we design the stubborn window for LTCP.
- We propose LTCP to handle packet reordering and link degradation in packet spraying circumstance. LTCP combines lazy trigger and stubborn window to improve the robustness and throughput of TCP over packet spraying in DCNs.
- We conduct experiments on NS2 to evaluate the performance of LTCP. Simulation results show that LTCP is robust and high-throughput, and improves 2.4 times throughput compared to existing TCP in cases of packet reordering and link degradation.

The rest of this paper is organized as follows. Section 2 briefly reviews multipath schemes and existing TCP schemes over packet spraying. Section 3 introduces our observations on link degradation. Section 4 describes the design of LTCP. Section 5 gives simulation results, and Section 6 concludes the paper.

2. Related work

In this section, we first review existing multipath schemes in DCNs. We explain why packet spraying outperforms other schemes and point to the problems of packet spraying. We also introduce existing TCP schemes over packet spraying, and explain why they are incompetent.

2.1. Multipath schemes in DCNs

There are two categories of multipath routing schemes: per-flow routing and per-packet routing.

2.2. Per-flow routing

This category is typically ECMP. ECMP is to decide the forwarding port by hashing the TCP 5-tuple. So every packet belonging to the same flow takes the same path to the destination and packet reordering cannot occur. However, due to hash collisions, some flows may be assigned to the same path, which leads to other paths idle. A recent study [4] shows that ECMP can only utilize 40–80% network capacity.

Hedera [4] was proposed to overcome ECMP's shortage. By dynamic scheduling long flow among link-disjoint paths, Hedera

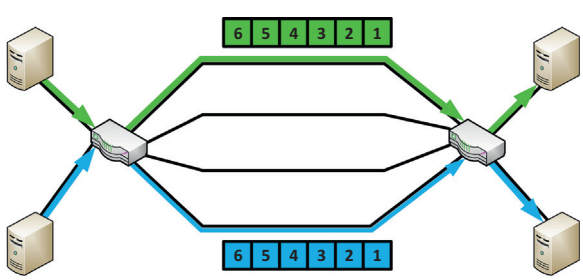
achieves better utilization of multiple paths than ECMP. But such centralized solution faces serious scalability challenge.

MPTCP [5] divides a TCP flow into several subflows. Each subflow owns a congestion window and uses ECMP to select its path independently. By concurrently exploring multiple paths and dynamical load balancing them, MPTCP achieves higher throughput and better robustness than ECMP. However, MPTCP is designed to continuously move traffic from congestion path to light-load one. Such design in itself indicates imbalanced load and non-optimal utilization of multiple paths. Unlike MPTCP which runs over ECMP, the LTCP proposed in this paper runs over packet spraying, which is a totally different multipath routing scheme.

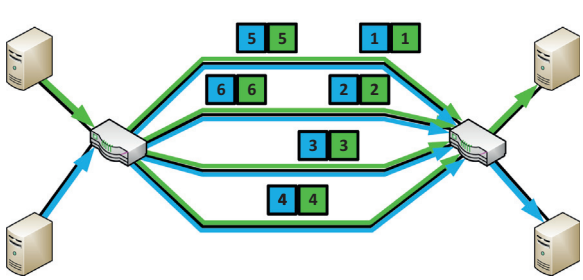
The fundamental flaw of these schemes is that they all take flow as the basic unit to route. However, sometimes it is hard to find a feasible solution. For example, given three flows, there is no way to evenly assign them on two paths. Flow splitting scheme, such as LocalFlow [13], could address this problem and achieve higher network utilization. But LocalFlow performs fine-grain flow measurement and multi-resolution splitting on switches, which may require support from programmable switch.

2.3. Per-packet routing

This category is typically packet spraying. Packet spraying is to evenly distribute packets of the same destination to multiple downstream paths, achieving perfect load balancing and full utilization of multiple paths. There are two kinds of packet spraying: Random packet spraying (RPS) is to distribute packets randomly among multiple paths, while RRPS [7] in a round-robin manner. Fig. 1 illustrates the comparison of ECMP and RRPS. We see that flows over ECMP occupy two paths and packets are transmitted orderly along them, leaving the other two paths idle. On the contrary, flows over packet spraying utilize all paths and packets are evenly distributed on them. We can easily figure out that packet spraying is much better than ECMP in both load balance and utilization of multiple paths. In addition, packet spraying has been widely implemented in commodity switch for many years [7]. So its throughput gain is realistic and can be easily achieved by configuring switch.



(a) ECMP causes load imbalance and low utilization of multiple paths



(b) RRPS achieves full utilization of multiple paths

Fig. 1. Comparison of ECMP and RRPS.

However, there are still two issues with packet spraying. First, packets sprayed among multiple paths may experience different latency and result in packet reordering. It is well known that packet reordering will trigger TCP congestion avoidance and lower throughput. Second, link degradation occurs frequently, and TCP throughput declines sharply if any link among multiple paths degrades. A recent study [5] shows that flows passing degraded link only achieve 10% throughput. Therefore, link degradation has been considered as one of the major reasons to keep packet spraying out of the practical applications in DCNs.

Dixit et al. [6] argue that due to symmetrical topology, packet reordering rarely occurs in DCNs. But they still point to that a robust TCP (e.g. TCP–NCR) is desirable. They also propose a switch-based scheme to handle packet reordering caused by link failure. But they do not consider the issue of link degradation. In their scheme, flows directly impacted by link failure are suppressed to make sure that they will not disturb other flows. In our scheme, no flow is suppressed and we do not attempt to avoid packet reordering. Instead, we make all flows robust to packet reordering and link degradation.

DeTail [12] is a load-aware packet spraying scheme to achieve per-packet adaptive load balancing. Switches use the congestion information obtained from port buffer occupancies to dynamically select port for every packet. Such scheme could handle link degradation. But it requires modification on the TCP stacks of both switches and end hosts, so it is not easy to be implemented in practices.

Digit-Reversal Bouncing (DRB) [14] is similar with RRPS. In our opinion, RRPS and DRB should achieve the same performance, because they distribute packets to multiple paths in the same order. Take the FatTree(4,3) (3 layer FatTree topology built with switches of 4 ports) in Fig. 4 for example. Given an inter-pod flow, both of them distribute packets to the core switches in the same order. We see that packet 1 is sprayed to C1, packet 2 is sprayed to C3, packet 3 is sprayed to C2, and packet 4 is sprayed to C4. However, DRB is a complex scheme, which requires techniques such as packet encapsulation and source-routing. While RRPS used in this paper is a simpler scheme implemented in commodity switches for years.

2.4. Existing TCP schemes over packet spraying

In this subsection, we briefly review existing TCP schemes over packet spraying, and explain why they are incompetent.

TCP-Reno [8] is the most widely implemented TCP scheme with AIMD (Additive Increase Multiplicative Decrease). The congestion avoidance of TCP-Reno works as follows. First, the sender accumulates duplicate ACKs it received. Second, if there are three accumulated duplicate ACKs, the sender triggers congestion avoidance. Third, during the period of congestion avoidance, the sender halves congestion window and retransmits the lost packet. Finally, the sender enters the fast recovery phase.

TCP-Reno is easy to be disturbed by packet reordering for two reasons. First, we see that the threshold to trigger congestion avoidance is quiet small (only three duplicate ACKs). Second, TCP cannot distinguish packet reordering from congestion. Both packet loss and disordered packets cause duplicate ACKs, which are considered as TCP congestion signals.

TCP-NewReno [9] and TCP–SACK [10] are improved variants of TCP-Reno. However, their improvements focus on designing better fast recovery mechanism. Their congestion avoidance algorithms are very similar to TCP-Reno's. So they also suffer from packet reordering.

Based on TCP–SACK, a robust TCP is proposed. TCP–NCR [11] uses a dynamic threshold to trigger congestion avoidance, and the threshold is usually much bigger than three duplicate ACKs. In this way, TCP–NCR waits more time for the delayed packets, hence mitigates the impact from packet reordering. LTCP also uses TCP–NCR's idea to handle packet reordering, and we call it lazy trigger. However, as TCP over packet spraying faces two challenges from packet reordering and link degradation, such design is incompetent. Our experiment in Section 5 shows that TCP–NCR can only tolerate packet reordering in certain degree, and is also helpless to link degradation.

DCTCP [3] is a recently proposed TCP variant which aims at low latency transmission in datacenter. It keeps all queues short by continuously adjusting data transmission rate according to Explicit Congestion Notification (ECN) from network. Running DCTCP over packet spraying can further improve the latency performance, as traffic is evenly distributed among all paths. However, DCTCP has not carefully considered the challenges in packet spraying circumstance. Our experiment in Section 5 shows that packet reordering and link degradation can largely depress its performance. Unlike DCTCP, the LTCP proposed in this paper aims at reliable transmission over packet spraying. It addresses the two issues above and achieves acceptable throughput shown by our experiments. In addition, LTCP and DCTCP can be complementary. LTCP reacts to duplicate ACKs while DCTCP reacts to ECN. Their functions are not conflicting and can be merged, but we leave this for future work.

3. Link degradation

In this section, we first explore the impact of link degradation using simulation experiments, and then characterize the packet loss induced by link degradation.

3.1. Problem statement

1000BASE-T [15] requires two things to provide 1Gbps bandwidth. First, all four twisted-pair of Cat5e or Cat6 cables must work normally for simultaneous transmission in both directions. Second, auto-negotiation [16] must be turned on as it determines the Master–Slave relationship between the PHYs at each end of the link.

Link degradation occurs in such circumstance. If one RJ45 conductor fails to seat properly, switches will auto-negotiation to 100 Mb/s, which only requires two twisted-pair cables [5,17]. When such link degradation happens, multiple paths are not equal-cost paths anymore. The bandwidth of degraded link is only 10% of the others. While packet spraying is a load-unaware scheme, which could only evenly distribute traffic on all downstream paths. Compared to packets passing normal links, the packet sprayed to a degraded link experiences 10 times transmission delay. The subsequent packets behind it could reach destination before it could. When these subsequent packets feed back three duplicate ACKs, congestion avoidance is triggered and TCP throughput declines. Fig. 2 illustrates this procedure. For the purpose of demonstration, the congestion window is initialized to 6 packets. The packet 1 and packet 5 jam on the degraded link. While packet 2, packet 3 and packet 4 reach destination before they could and feed back three duplicate ACK 0. When the sender receives the third duplicate ACK 0, it considers packet 1 is lost and performs congestion avoidance.

Furthermore, the impact of link degradation is permanent unless fixing the link. As long as packets are sprayed to the degraded link, TCP falls into continuous congestion avoidance or even timeout.

3.2. The impact of link degradation

We conduct several experiments on packet-level simulator NS2 to examine TCP's performance in DCN with degraded links

3.2.1. Throughput

Link degradation severely reduces TCP throughput. We drop a link connecting the core switch and Pod 1 from 1 Gb/s to 100 Mb/s in FatTree(4,3), and then run long TCP flows in a random permutation matrix (similar to [6]). In such a setup, all servers are randomly paired as sender and receiver. Every server is either a sender or a receiver of exactly one flow. The sender sends traffic to its receiver for the whole duration of the experiment. Flows passing Pod 1 are impacted by the degraded link. We measure the average throughput of the impacted flows as a percentage of the ideal throughput. The experiment results are show in Fig. 3. We see that all TCP schemes (TCP-Reno, TCP–SACK and DCTCP) suffer from link degradation and their throughputs drop to 15–32%. When we expand the topology to FatTree(6,3) and FatTree(8,3), the throughput increases. This is because the damage of single degraded link is shared by more TCP flows. However, given a fixed rate of link degradation, bigger topology may also induce more degraded links.

3.2.2. Scope

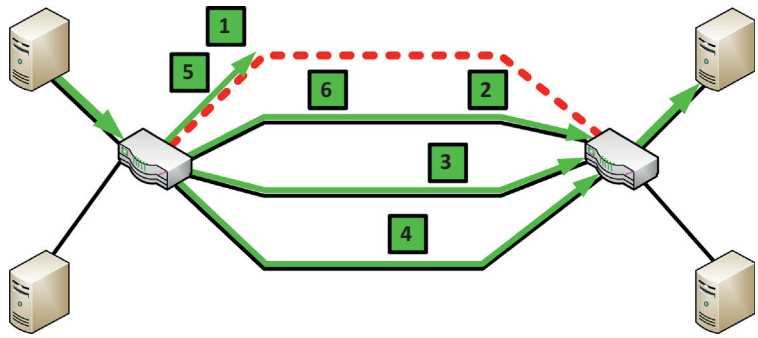
A few degraded links can impact a large number of servers in DCN and decrease their TCP throughput. This is because the routing mechanism of packet spraying enlarges the scope of impacted servers. Unlike ECMP, which assigns one flow on one path, packet spraying distributes one flow on all downstream paths. So every path loads all flows traveling in the same direction, and any degraded link will damage all these flows. A degraded core link will impact all servers in the pod it connected with; a degraded aggregation link will impact all servers in the rack it connected with. For example, in Fig. 4, the degraded link between A1 and C1 will impact all inter-pod flows connecting with server 1 – server 4 in Pod 1. And the degraded link between A3 and T3 will impact all flows connecting with server 5 and server 6.

We want to know how many servers will be impacted in a large topology. So we setup an experiment in a FatTree(48,3). In such topology, there are in total 27648 servers and 82944 links. Given certain rate of link degradation, we randomly generate degraded links and calculate the impacted servers. The experiment results are shown in Fig. 5. The green line demonstrates that just a few degraded core links can impact a large number of servers. For example, when the rate is just 0.1%, around 41% servers will suffer throughput decline. When the rate increases to 1%, almost all servers are impacted. As degraded links between servers and TOR (Top of Rack) switches only directly impact servers they connected to, we ignore their impact in this experiment.

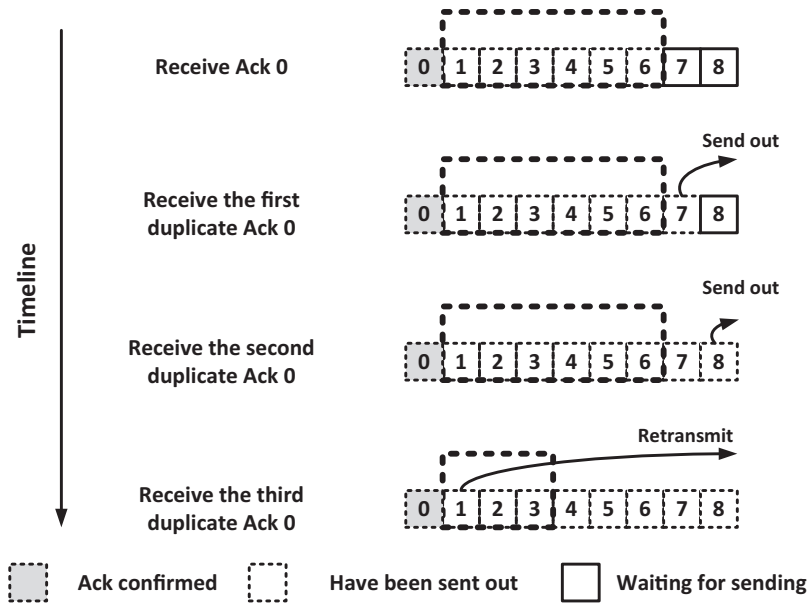
3.3. Characteristic of packet loss

When running TCP over packet spraying in a multipath network with degraded links, the sender may detect three cases of packet loss. Case 1: Discrete packet loss occurs in a batch of packets. Case 2: Successive packet loss occurs in a batch of packets, which means a packet is sprayed to a degraded link, and the following one is sprayed to another degraded link. Case 3: No packet loss, which indicates that the flow does not encounter any degraded link on its paths.

Our intuition is that the sender may often encounter case 1, while case 2 should rarely occur. As for case 3, it is determined by the rate of link degradation. To validate our hypothesis, we traverse all multiple paths in a FatTree topology to see how often these cases happen. As multiple paths only exist in aggregation layer



(a) Packet 1 and packet 5 are delayed on the degraded link



(b) Packet 2 - 4 feed back duplicate ACKs and trigger TCP congestion avoidance

Fig. 2. Link degradation causes continuous congestion avoidance.

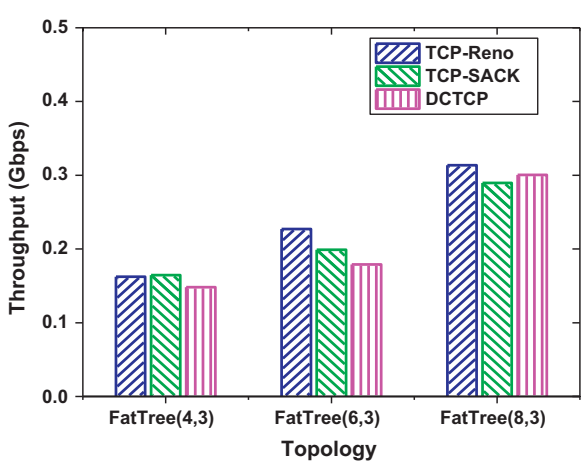


Fig. 3. TCP throughput in case of single degraded core link.

and core layer, we could consider TOR switches as the start point and end point of TCP flows over multiple paths. When a TCP connection is established, packets are transmitted from one TOR

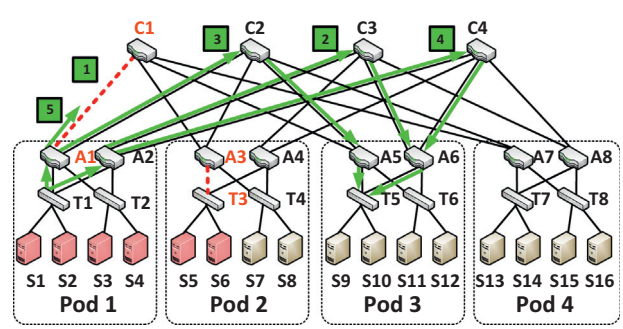


Fig. 4. The impact of link degradation in FatTree(4,3).

switch to another. Assuming that all switches use RRPS to route packets, some packets may be sprayed to a degraded link and get dropped. We log these packets' IDs and then compute the interval between every pair of adjacent packet loss. Interval 0 indicates successive packet loss. Interval greater than 0 indicates discrete packet loss. And interval of infinite indicates no packet loss. Take switch pair of T1 and T5 in Fig. 4 for example. For a complete round to traverse four paths between them, 5 packets are needed. Packet 1 – packet 5 will traverse switches in the following way:

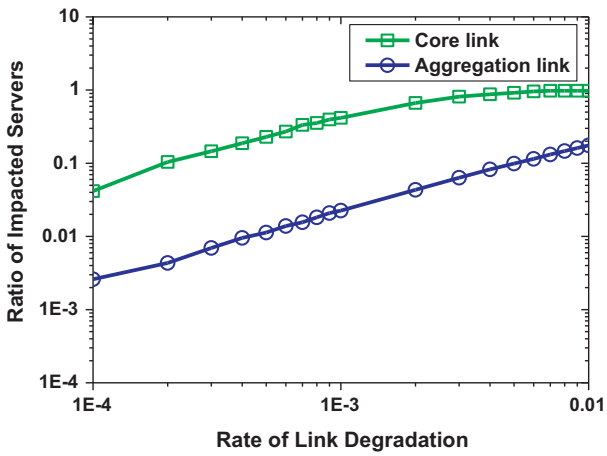


Fig. 5. Impacted server ratio under different rate of link degradation.

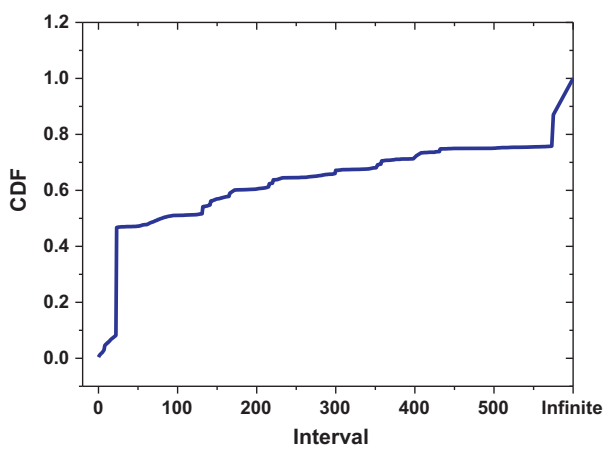


Fig. 6. Distribution of interval in FatTree(48,3).

Packet 1	T1→A1→C1→A5→T5	packet loss
Packet 2	T1→A2→C3→A6→T5	**Interval 3 indicates discrete packet loss**
Packet 3	T1→A1→C2→A5→T5	
Packet 4	T1→A2→C4→A6→T5	
Packet 5	T1→A1→C1→A5→T5	packet loss again

As the link between A1 and C1 is degraded, packet 1 and packet 5 are lost, so we log both of them. Then we could infer that the interval between them is 3. Such interval clearly indicates a discrete packet loss. On the contrary, if there is another degraded link between A2 and C3, and packet 2 is lost too. We log packet 1, packet 2 and packet 5, and the corresponding interval is 0 and 2. The interval 0 clearly indicates successive packet loss caused by packet 1 and packet 2.

We repeat the above procedure for every pair of TOR switches in FatTree(48,3) with 0.1% degraded links. The experiment result is showed in Fig. 6. We carefully examine the frequency of interval 0, which indicates successive packet loss, and get the value of only 0.4%. While the interval greater than 0, which indicates discrete packet loss, accounts for 86.6%. And the interval of infinite, which indicates no packet loss, accounts for 13%. So the experiment result proves our hypothesis that degraded links in a multipath network stand a good chance of causing discrete packet loss in a batch of packets. In addition, we also found that interval clusters around 23 packets and 575 packets. Such phenomenon is induced by the characteristic of FatTree topology. In FatTree(48,3), there are 24 paths for intra-Pod flow, and 576 paths for inter-Pod flow. So a single degraded aggregation link will cause discrete packet loss with interval 23. And a single degraded core link will cause discrete packet loss with interval 575.

Algorithm 1 Congestion Avoidance mechanism of LTCP

```
If receive a Duplicate Ack
  If Duplicate Acks == Max(3, congestion window) then
    Retransmit the lost packet;
    If detect successive packet loss then
      Halve congestion window
    End if
    Enter fast recovery phase;
  Else
    Send out one packet
  End if
End if
```

4. Lazy TCP

In this section, we describe LTCP's design in detail. We present how to handle packet reordering and link degradation. We then present how to cut down the overhead induced by our design.

4.1. Handle packet reordering and link degradation

The basic idea of LTCP is that TCP should not backoff quickly when receiving congestion signals, as these signals can be easily caused by non-congestion events in packet spraying circumstance. Internet-oriented TCP is sensitive to congestion signals that it halves data transmission rate as soon as receiving it. Such design is reasonable as TCP flows on Internet are usually single-path. In such circumstance, packets belonging to the same flow are transmitted orderly along one path, and packet reordering rarely occurs. So it is very possible that duplicate ACKs do signify packet loss caused by congestion, and TCP should backoff in return. However, in packet spraying circumstance, packets belonging to the same flow are transmitted along different paths, and duplicate ACKs may be caused by packet reordering or link degradation. So TCP should not backoff quickly in response to these duplicate ACKs.

LTCP combines two ideas to perform congestion avoidance in a lazy way. The first one is called lazy trigger, which could mitigate the impact of packet reordering. The second one is called stubborn window, which could eliminate the impact of link degradation. The overall congestion avoidance algorithm is showed above. We first explain the two ideas and then illustrate the whole procedure.

The core idea of lazy trigger is that TCP can mitigate the impact of packet reordering by delaying congestion avoidance. The time to perform congestion avoidance is determined by the number of duplicate ACKs required to trigger it, which is usually be considered as a threshold. The greater the threshold, the later the sender performs congestion avoidance. Common TCP, such as TCP-Reno, TCP-Newreno and TCP–SACK, sets the threshold to three duplicate ACKs, which may be easily reached in case of packet reordering. As the number of duplicate ACKs also indicates the degree of packet reordering, TCP could increase the threshold to ignore minor packet reordering. For example, one duplicate ACK indicates that a packet is delayed and the subsequent one reaches destination before it could. Five duplicate ACKs indicate that a packet is largely delayed and the subsequent five packets reach destination before it could. If the threshold is set to six duplicate ACKs, none of the two events above can trigger TCP congestion avoidance. In other words,

by increasing the threshold of triggering congestion avoidance, TCP can tolerate more severe packet reordering and prevent unnecessary congestion avoidance. The mechanism of lazy trigger is show in line 2 of algorithm 1. Note that the threshold is set equal to the congestion window. This is because when such condition holds, the sender has spent at least two RTT (round-trip-times) on waiting for the delayed packet. This period of time is enough for most of disordered packets, because normally they should feed back ACKs at the end of the first RTT. Later we will illustrate this mechanism with an example.

The core idea of stubborn window is that TCP can eliminate the impact of link degradation by maintaining data transmission rate in case of discrete packet loss. When performing congestion avoidance, existing TCP schemes directly halve congestion window to decrease data transmission rate. LTCP first identifies the state of packet loss, and then takes proper action. In case of discrete packet loss, the congestion window stays the same. In case of successive packet loss, the congestion window is halved (lines 4–6 of algorithm 1). We make such design for two reasons. First, our observation in previous section shows that link degradation stands a good chance of causing discrete packet loss among a batch of packets. So maintaining congestion window in such case could effectively eliminate the impact of link degradation. Second, it is appropriate to consider successive packet loss as congestion signals. DropTail, which is the simplest queue, could easily cause successive packet loss. In fact, such phenomenon is called lockout [18]. When lockout happens, a few flows monopolize queue space, preventing other flows from entering the queue. So these flows experience successive packet loss. Moreover, pFabirc [19], which is the best known prior approach for minimizing flow completion times in DCN, could cause successive packet loss too. pFabric performs strict flow scheduling on switches. When flow of high priority occupies the link, flows of low priority will either wait in the queue or experience successive packet loss.

Now we illustrate how algorithm 1 works. Every time the sender receives a duplicate ACK, it resolves the SACK (Selective Acknowledgment, which is embedded in the option field of ACK) to know which packet has been received, which packet is still on the fly. If the number of duplicate ACKs does not reach the threshold (line 2 of algorithm 1), the sender sends out one packet to keep the traffic flowing. If the threshold is reached, the sender performs congestion avoidance. The sender scans packets in the congestion window to identify the lost one. If it finds discrete packet loss, it maintains the congestion window. On the contrary, if it finds successive packet loss (lines 4–6 of algorithm 1), it halves congestion window. In either case, the lost packet is retransmitted and LTCP enters fast recover phase.

Fig. 7 illustrates LTCP's congestion avoidance in case of different packet loss. For the purpose of demonstration, the congestion window is initialized to 6 packets. In subgraph (a1), the sender receives packet 0, and it sends out packet 1–6 in its congestion window. After one RTT, in subgraph (a2), the sender receives the first duplicate ACK 0, and it sends out packet 7 to keep traffic flowing (line 9 of algorithm 1). In subgraph (a3), the sender receives more duplicate ACKs, and it sends out packets 8–11 in response. Finally, in subgraph (a4), after another RTT, the sender receives the sixth duplicate ACK 0, which triggers congestion avoidance. Note that the sender has spent two RTT on waiting for the packet 1. This period of time is enough for most of disordered packets, because normally they should feed back ACKs at the end of the first RTT. Now the sender scans packet in the congestion window to identify the lost one. As it identifies discrete packet loss (packet 1), the sender maintains congestion window and retransmits packet 1. On the contrary, subgraphs (b1) – (b4) depict what LTCP does in case of successive packet loss. The congestion window is halved and packet 1 is retransmitted.

4.2. Reduce overhead of packet loss

Stubborn window may induce overhead of packet loss. Lockout phenomenon does not always occur in DropTail queue. Switches may also use other queues, such as RED [20] and BLUE [21]. So sometime, discrete packet loss dose signify congestion. In that case, LTCP will mistake discrete packet loss as the signal of link degradation. And it will not backoff properly and continue to send out packets in high speed. It is very possible that these packets will be dropped on the bottleneck link as the network is already congested. We consider the extra packet loss on the bottleneck link as the overhead of LTCP. It should be noted that such overhead can be completely eliminated by combining LTCP with congestion control techniques such as DCTCP [3] and ICTCP [22]. However, if such techniques are not used, LTCP does induce overhead, and we propose a simple scheme to reduce it.

In order to reduce the overhead, we set an upper bound for LTCP's congestion window. We make such design for two considerations. First, the characteristic of DCNs suggests that TCP could set an upper bound for its congestion window. DCNs (e.g. Fattree) are often built with uniform links, and hosts own the same uplink bandwidth, which indicates that the max data transmission rate is deterministic. Therefore it is unnecessary for TCP to increase congestion window if it has reached the max data transmission rate. Second, limited congestion window ensures that TCP flows form limited queue, hence reduces packet loss on the bottleneck link. As congestion window directly determines the number of packets on the fly, it also determines the length of queue on the bottleneck link. Large congestion window leads to long queue and packet loss, while small congestion window leads to short queue. As Fig. 8 shows, congestion window without limit will keep on increasing until packet loss occurs, which indicates that the queue is already full. On the contrary, flows with limited congestion window form a short queue and no packet is dropped. In addition, short queue also benefits time sensitive flows for lower latency.

The upper bound should be small enough to reduce packet loss while large enough to reach theoretical line rate, which is determined by the theoretical RTT. In FatTree topology, there are three kinds of flows and they have different theoretical RTTs. These flows are intra-rack flow (2 hops), intra-pod flow (4 hops) and inter-pod flow (6 hops). The more hops, the larger the RTT, and the larger congestion window is needed to stature network and reach theoretical line rate. For intra-rack flow, we set the upper bound to its theoretical line rate. For other two kinds of flow, we set their upper bounds to double theoretical line rate. For example, in our experiment setup in Section 5, the theoretical RTT of intra-rack flow is 29us, which suggests that a single flow with a congestion window of about three packets could fully stature network and reach line rate. So we set the upper bound for intra-rack flow to three packets. The theoretical RTT of inter-pod flow is 86us, which suggests that a single flow with a congestion window of about 7 packets could reach line rate, but we set the upper bound for inter-pod flow to 13 packets. We make such setting because inter-pod flow and intra-pod flow travel multiple paths where packet spraying functions, hence a congestion window beyond line rate can provide some budget in case of link degradation. Such setting is also feasible as ip address in FatTree are usually location-related [1], hence a sender can learn the location of its receiver before transmission and choose proper upper bound.

5. Simulations

In this section, we conduct experiment to evaluate LTCP's performance. We seek answers for three questions. Can LTCP maintain high throughput in cases of packet reordering and link

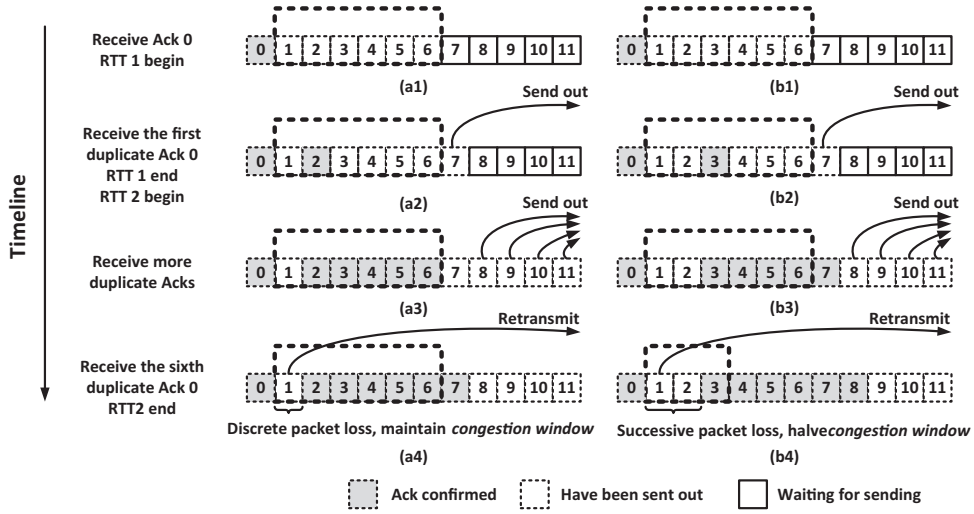
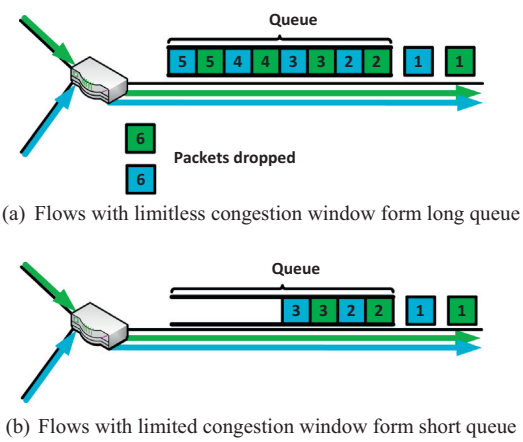


Fig. 7. LTCP's congestion avoidance in case of different packet loss.



(a) Flows with limitless congestion window form long queue

(b) Flows with limited congestion window form short queue

Fig. 8. Limit max congestion window to reduce packet loss on the bottleneck link.

degradation? Can limited congestion window successfully reduce LTCP's overhead? Can LTCP works well in Incast [23] scenario?

We implement LTCP in NS2 by modifying about 100 lines of code on TCP–SACK. The main changes focus on function *dupaction()*. Conventional TCP uses this function to retransmit the lost packet and halve congestion window. We insert a statement *if* between them. When performing congestion avoidance, LTCP always retransmits the lost packet, but only halves congestion window in case of successive packet loss.

5.1. Handle packet reordering and link degradation

In this subsection, we setup three scenarios in FatTree(8,3) to evaluate LTCP's performance in abnormal circumstances.

(1) *Packet reordering:* We use error model to randomly delay 2% packets on every host's uplink, generating reordering on purpose. We choose such parameter because a recently study [6] observes about 2% disordered packets naturally exist in their experiment.
(2) *Link degradation:* We drop a single link between core and Pod 1 from 1 Gbps to 100 Mbps. All flows passing Pod 1 are impacted.
(3) *Packet reordering and link degradation:* Both scenario (1) and scenario (2) coexist.

The experimental parameters are shown in the Table 1. We measure the throughput of long lived TCP flows with a permutation matrix traffic pattern (similar to [6]). In such a setup, each host in the topology is either a sender or a receiver of exactly one TCP flow. Every sender is randomly paired with a receiver in another pod. All senders start transmission at the same time and last for 10 s. All TCP schemes use a fixed timeout value (1000 μs) as pFabric [19] does. We also repeat the experiment on different timeout value (300 μs, 500 μs and 1500 μs) and get similar results.

We compare four schemes. TCP-Reno: As TCP-Reno, TCP-Newreno and TCP–SACK perform similarly, we choose TCP-Reno for comparison since it is the most widely used TCP scheme. TCP–NCR: It is robust to packet reordering, but it cannot tolerate frequent packet loss caused by link degradation. The comparison with TCP–NCR illustrates the significance of LTCP's stubborn window. DCTCP: It is designed for low latency transmission in datacenter. The comparison with DCTCP shows that different design goals result in different performances. LTCP: It is designed to handle both packet reordering and link degradation in packet spraying circumstance. As all flows are inter-pod flow, the upper bound of LTCP's congestion window is set to 13 packets.

Fig. 9 depicts the comparison of these schemes in scenario (1). When the randomly selected packets are delayed for 100–200 μs, the throughput of DCTCP and TCP-Reno declines to 69–78%; TCP–NCR and LTCP are almost not impacted, and they maintain 97–99% throughput. When the delay increases to 300 μs, some packets trigger TCP–NCR's congestion avoidance, hence its throughput declines to 78%. When the delay further increases to 400 μs, DCTCP, TCP-Reno and TCP–NCR get similar average throughput (about 70%). This is because the largely delayed packet causes too many duplicate ACKs, which easily triggers TCP–NCR's congestion avoidance. In other words, although TCP–NCR is robust to packet reordering, it still cannot tolerate such large delay. From

Table 1
Parameters for experiment.

Traffic pattern	Permutation, long flow
Link bandwidth	1 Gbps
Propagation delay	1 μs
Inter-pod RTT	86 μs
Intra-rack RTT	29 μs
DropTail queue length	80 packets
Packet size	1500 B

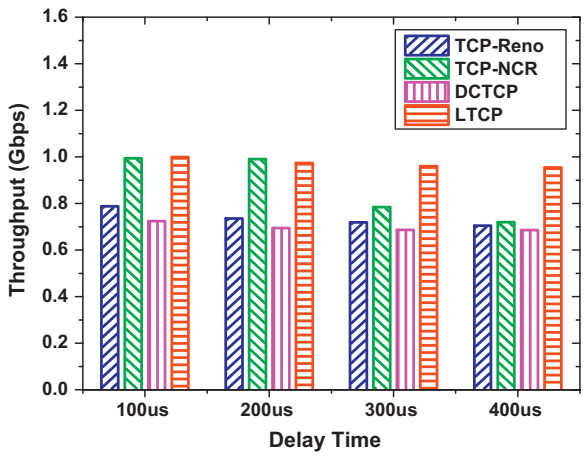


Fig. 9. Throughput comparison in case of packet reordering.

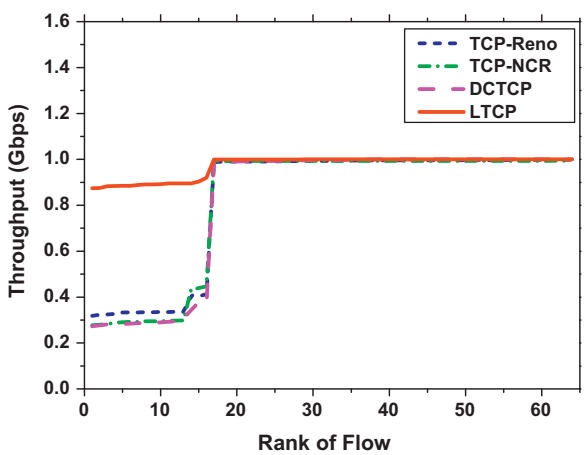


Fig. 10. Throughput comparison in case of link degradation.

the perspective of the sender, these delayed packets are actually equivalent to be lost. LTCP is designed to be unresponsive to discrete packet loss, hence it ignores those delayed packets and still maintains 95% throughput. We also measure the standard deviation of their throughputs and find that LTCP's performance is very stable. For example, when the delay time is 400 μs, LTCP's standard deviation is only 0.008. While the standard deviation of other schemes distributes from 0.014 to 0.018. Experiment in this scenario demonstrates that LTCP is robust to packet reordering.

Fig. 10 depicts the comparison of TCP schemes in scenario (2). The throughputs of individual flows are sorted in increasing order. The 16 flows passing through Pod 1 are impacted by link degradation. We can see that both TCP-Reno, TCP–NCR and DCTCP perform poorly, and their throughputs distribute from 27% to 44%. This is because the degraded link continuously generates congestion signals, which force these TCP schemes to reduce data transmission rate. On the contrary, LTCP still maintains 87–91% throughput, which is about 2.5 times average throughput compared to DCTCP. This is because LTCP carefully analyzes the congestion signal and distinguishes link degradation from congestion. Flows in Pod 2 – Pod 8 are not impacted by link degradation, hence all schemes achieve ideal throughput. Experiment in this scenario demonstrates that LTCP achieve the same throughput as existing TCP schemes in normal circumstances, but much higher throughput in case of link degradation.

Fig. 11 depicts their performance in scenario (3). The 16 flows passing through Pod 1 are impacted by both packet reordering and link degradation. LTCP can still maintain 76–84% throughput, which is about 2.4 times average throughput compared to DCTCP. Flows in other Pods are only impacted by packet reordering (random 2% packets are delayed for 100 μs), hence all schemes get similar throughput as scenario (1). Experiment in this scenario demonstrates that LTCP is robust to both packet reordering and link degradation.

5.2. Overhead of packet loss

In this subsection, we conduct experiment to validate whether a limited congestion window can reduce LTCP's packet loss on the bottleneck link. As mentioned in Section 4, the overhead can be completely eliminated by combining LTCP with congestion control techniques such as DCTCP [3] and ICTCP [22]. However, if these techniques are not used, we could limit the max congestion window to reduce LTCP's overhead.

We setup a many-to-one traffic pattern in FatTree(8,3). In such setup, several inter-pod long flows simultaneously swarm into a server in Pod 1 and last for 10 s. For the purpose of illustration, we set the queue length to 150 packets and the timeout value to 2000 μs accordingly. We vary the number of senders to measure the following three experimental indicators: Total packets confirmed by receiver, which indicates LTCP's ability to efficiently transmit during congestion. Total packets dropped on the bottleneck link, which reflects the overhead of LTCP. Average queue length of the bottleneck link, which resolves the reason of packet loss on the bottleneck link.

We compare three schemes. TCP–SACK: We do not take TCP-Reno as the baseline here. This is because our LTCP utilizes SACK and performs much better than TCP-Reno in case of congestion. So we choose TCP–SACK for a fair comparison. LTCP-U: We setup a LTCP with an Unlimited congestion window. LTCP: As all flows are inter-pod flow, the upper bound of LTCP's congestion window is set to 13 packets.

Fig. 12 depicts the comparison of LTCP-U vs. TCP–SACK and LTCP vs. TCP–SACK. We can see that all TCP schemes successfully transmit the same among of packets. However, the packet loss on the bottleneck link differs. LTCP-U does not limit congestion window, the sender increases congestion window rapidly and sends out too many packets. As a result, LTCP-U drops much more packets on the bottleneck link than LTCP. For example, when there are just 6 senders, LTCP-U causes 3 times packet loss compared to TCP–SACK, while LTCP with an upper bound always achieves zero packet loss. This is because the limited congestion window also limits the total number of packets on the fly, hence the queue of

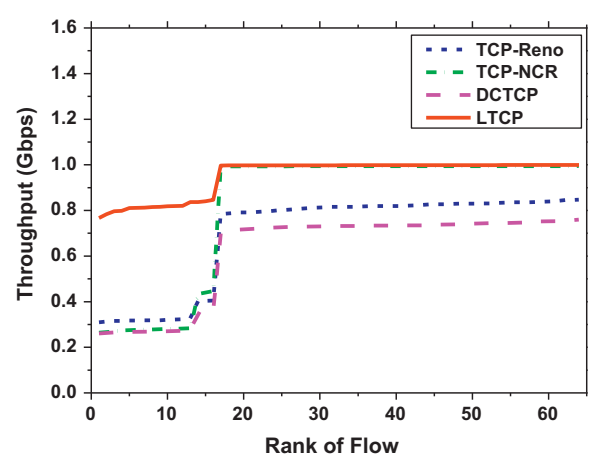


Fig. 11. Throughput comparison in cases of packet reordering and link degradation.

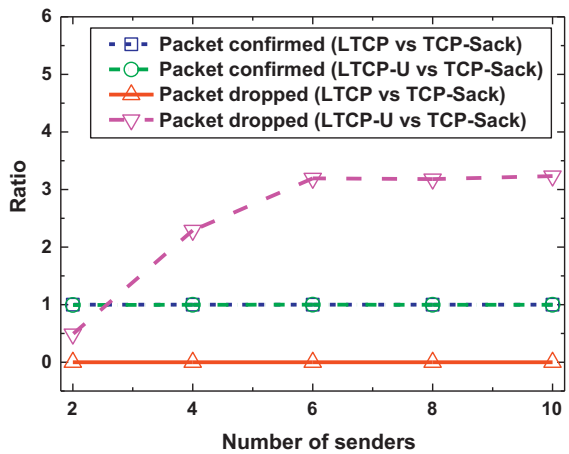


Fig. 12. Performance comparison: LTCP-U vs. TCP–SACK and LTCP vs. TCP–SACK.

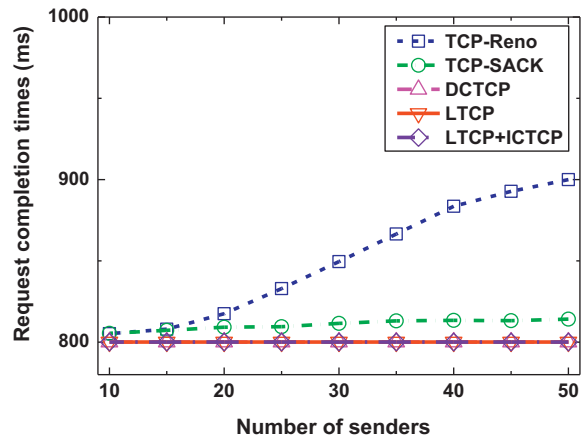


Fig. 14. Request completion times comparison in Incast scenario.

bottleneck link can hold these packets and no packet will be dropped. For example, when there are 10 senders, the total number of packets on the fly is about 130, while the queue of the bottleneck link can hold 150 packets. Fig. 13 clearly resolves the experiment results in Fig. 12. Since LTCP limits the max congestion window, the queue of the bottleneck link increases linearly with the number of senders. While LTCP-U and TCP–SACK form a long queue with just 2 senders. The experiment result is also a strong evidence for our schematic diagram Fig. 8.

5.3. Comparison with existing schemes in incast scenario

In this subsection, we evaluate LTCP's performance in Incast scenario. We show that although LTCP is not designed for Incast, it can also largely mitigate the Incast problem by combining with existing techniques [22,24].

Incast [23] is one of the crucial issues [25] that severely decreases TCP throughput in DCN. In Incast scenario, multiple senders concurrently transmit packets to a single receiver through a switch. The aggregated packets then overwhelm the switch's buffer. So TCP timeout occurs and flow experiences long idle time, leading to low throughput.

To address the Incast problem, several schemes have been proposed in recent years. A recent study [24] has suggested that setting a small timeout value can reduce the bandwidth waste during timeout. ICTCP [22] is placed at the receiver side to advertise senders the proper size of congestion window for avoiding packet loss on the bottleneck. DCTCP [3] also has effect on Incast as it tries to keep queue short by continuously adjusting packet transmission rate according to ECN feedback. Different from above schemes, our proposed LTCP is not designed specifically for the Incast problem. Even though, we show experimentally that LTCP can largely mitigate the Incast problem by combining with existing techniques such as small timeout value and ICTCP.

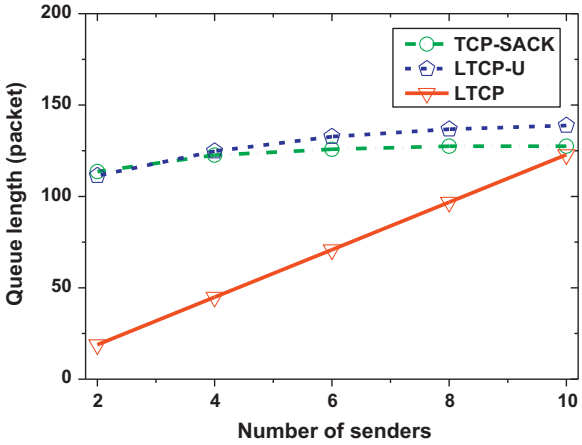


Fig. 13. Queue length comparison on the bottleneck link.

In the experiment, we setup an Incast scenario like previous work [19]. First, a receiver node requests a 100 MB file that is striped across N sender nodes. Then, each sender responds with 100 MB/N of data simultaneously. The request completes when all the individual flows have finished. Assume that the bandwidth of all links is 1Gbps. We derive that the theoretical request completion time is 800 ms (100 MB/1 Gbps). In practice, there are longer request completion times due to Incast. We compare five TCP schemes: TCP-Reno, TCP-Sack, DCTCP, LTCP and LTCP + ICTCP that is a combination scheme of LTCP and ICTCP. We set the switch's ECN threshold for DCTCP to 15 packets for optimal performance. We set the switch's queue length to 80 packets. The RTT is set to 29 μs. As all flows are intra-rack flow, the upper bound of LTCP's congestion window is set to three packets. We vary the number of senders to evaluate the performance of above different schemes.

Fig. 14 shows the request completion times. We can see that all five schemes perform fairly well, as we use a fixed small timeout value (1000 μs) like [19] which can greatly mitigate Incast [24]. TCP-Reno uses a simple congestion control algorithm that is known to be inapplicable to multiple packet loss in congestion window, so its request completion times is the longest one. TCP-Sack is an improved variant of TCP-Reno with better fast recovery mechanism, so its request completion times is shorter than TCP-Reno. LTCP is modified from TCP-Sack, and we have improved its timeout function. In case of timeout, TCP-Sack clears its scoreboard and retransmits packets, while LTCP maintains the scoreboard and only retransmits packets that have not been confirmed by the receiver. Additionally, as we have mentioned before, the upper bound of LTCP also reduces the probability of packet loss. So it is reasonable that LTCP outperforms TCP-Sack and achieves near-ideal request completion times. As for DCTCP, it avoids packet loss by rapidly adjusting the sending rate according to ECN feedback, so it is normal that it also achieves near-ideal request completion times. Since ICTCP actually takes over the congestion window of LTCP, LTCP + ICTCP can therefore achieve near-ideal request completion times.

Fig. 15 shows the traffic dropped on the bottleneck, which also clearly resolves the results in Fig. 14. The better the congestion

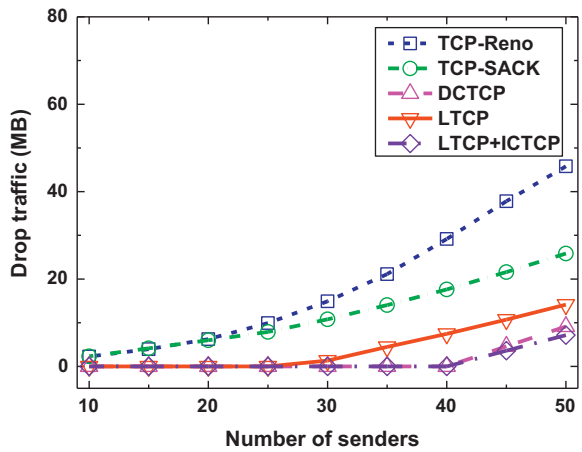


Fig. 15. Packet loss comparison in Incast scenario.

control algorithm, the less packet loss, and the less time to complete the transmission. So we see that TCP-Reno drops the most traffic. TCP-Sack drops less traffic for better fast recovery mechanism. LTCP drops zero traffic when the number of senders is no more than 25. While the number of senders exceeds 25, LTCP starts to drop packets. This is because the upper bound of LTCP limits the number of packet flighting in network. If the number of flighting packets is less than the queue length, no packet will be dropped. DCTCP tries to keep queue short, which also has effect on avoiding packet loss, so it achieves zero packet loss when the number of senders is no more than 40. The combination scheme of LTCP and ICTCP also achieves similar performance. In fact, as ICTCP actually takes over the sender's congestion window, combining ICTCP with other TCP schemes such as TCP-Reno and TCP-Sack can also produce the same result. Since the switch's queue length is set to 80 packets in the test, we see that although DCTCP and ICTCP have very good congestion control algorithm, they still suffer packet loss if there are too many senders.

6. Conclusion

This paper proposes LTCP, a novel TCP over packet spraying to achieve high throughput and robustness. The basic idea is that TCP over packet spraying should not back off quickly when receiving congestion signals, as these signals can be easily caused by packet reordering or link degradation. LTCP performs congestion avoidance in a lazy way. First, to mitigate the impact of packet reordering, LTCP delays the time to perform congestion avoidance, hence significantly reduces unnecessary congestion avoidance. Second, to eliminate the impact of link degradation, LTCP stubbornly maintains congestion window in case of discrete packet loss. Such design is based on our key observation that link degradation stands a good chance of causing discrete packet loss in a batch of packets. And it effectively eliminates the impact of link degradation.

We conduct experiments on NS2 to evaluate LTCP's performance. In case of packet reordering, LTCP maintains at least 95% throughput. In case of link degradation, LTCP achieves 2.5 times throughput compared to DCTCP. In both of the cases, LTCP still achieves 2.4 times throughput compared to DCTCP. We also evaluate LTCP's overhead. The results show that LTCP's overhead is acceptable. Moreover, LTCP is also used to mitigate the Incast problem by combining existing techniques. The results demonstrate that LTCP achieves near-ideal request completion times in Incast scenario.

Acknowledgments

This work is supported by the National Basic Research Program of China (973) under Grant 2012CB315805, the National Science Foundation of China under Grants 61173167, 61472130 and 61100171, the Prospective Research Project on Future Networks of Jiangsu Future Networks Innovation Institute under Grant BY2013095-1-05.

References

[1] M. Al-Fares, A. Loukissas, A. Vahdat. A Scalable, Commodity Data Center Network Architecture, in: SIGCOMM, 2008.
[2] A. Greenberg, N. Jain, S. Kandula, C. Kim, P. Lahiri, D. Maltz, P. Patel, S. Sengupta. VL2: A Scalable and Flexible Data Center Network, in: SIGCOMM, 2009.
[3] M. Alizadeh, A. Greenberg, D.A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, M. Sridharan. Data center tcp (dctcp), in: SIGCOMM, 2010.
[4] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, A. Vahdat. Hedera: Dynamic Flow Scheduling for Data Center Networks, in: NSDI, 2010.
[5] C. Raiciu, S. Barre, C. Pluntke, A. Greenhalgh, D. Wischik, M. Handley. Improving Datacenter Performance and Robustness with Multipath TCP, in: SIGCOMM, 2011.
[6] A. Dixit, P. Prakash, Y. C. Hu, R.R. Kompella, On the Impact of Packet Spraying in Data Center Networks, in: INFOCOM, 2013.
[7] Cisco. Per-packet load balancing. <http://www.cisco.com/en/US/docs/ios/120s/feature/guide/pplb.html>.
[8] K. Fall, S. Floyd, Simulation-based comparisons of tahoe, reno and sack tcp, SIGCOMM Comput. Commun. Rev. 26 (3) (1996) 5–21.
[9] S. Floyd, T. Henderson. The newreno modification to tcp's fast recovery algorithm. United States, 1999.
[10] M. Mathis, J. Mahdavi, S. Floyd, A. Romanow. TCP Selective Acknowledgment Options. Internet Engineering Task Force, RFC 2018, October 1996.
[11] Bhandarkar, Reddy, et al., Improving the Robustness of TCP to Non-Congestion Events. IETF RFC4653. August 2006.
[12] D. Zats, T. Das, P. Mohan, D. Borthakur, R. Katz. Detail: Reducing the flow completion time tail in datacenter networks, in: SIGCOMM, 2012.
[13] S. Sen, D. Shue, S. Ihm, M.J. Freedman. Scalable. Optimal Flow Routing in Datacenters via Local Link Balancing, in: CoNEXT, 2013.
[14] J. Cao, R. Xia, P. Yang, C. Guo, G. Lu, L. Yuan, Y. Zheng, H. Wu, Y. Xiong, D. Maltz. Per-packet load-balanced, low-latency routing for clos-based data center networks, in: CoNEXT, 2013.
[15] Gigabit Ethernet. <http://en.wikipedia.org/wiki/Gigabit_Ethernet#1000BASE-T>.
[16] IEEE 802.3af interpretation. <http://standards.ieee.org/findstds/interps/IEEE802.3af-2003interp-6.pdf>.
[17] Broadcom. Ethernet NIC FAQs. <http://www.broadcom.com/support/ethernet_nic/faq_drivers.php>.
[18] B. Braden, D. Clark, et al., Recommendations on Queue Management and Congestion Avoidance in the Internet. IETF RFC2309. April 1998.
[19] Mohammad Alizadeh, ShuangYang. pFabric: Minimal Near-Optimal Datacenter Transport, in: SIGCOMM, 2013.
[20] S. Floyd, V. Jacobson, Random early detection gateways for congestion avoidance, IEEE/ACM Trans. Juniper Networks Netw. 1 (4) (1993) 397–413.
[21] W. Chang Feng, K. Shin, D. Kandlur, D. Saha, The blue active queue management algorithms, Networking IEEE/ACM Trans. 10 (4) (2002) 13–528.
[22] Haitao Wu, Zhenqian Feng, Chuanxiong Guo, Yongguang Zhang, ICTCP: incast congestion control for TCP in data-center networks, Networking, IEEE/ACM Trans. 21 (2) (2013) 345–358.
[23] D. Nagle, D. Serenyi, A. Matthews. The Panasas ActiveScale Storage Cluster: Delivering Scalable High Bandwidth Storage, in: ACM/IEEE SuperComputing, 2004, pp. 53–62.
[24] V. Vasudevan, A. Phanishayee, H. Shah, E. Krevat, D. Andersen, G. Ganger, G. Gibson, B. Mueller. Safe and effective fine-grained TCP retransmissions for datacenter communication, in: SIGCOMM, 2009.
[25] Yan. Zhang, Nirwan. Ansari, On architecture design, congestion notification, TCP incast and power consumption in datacenters, IEEE Commun. Surv. Tutorials 15 (2013).